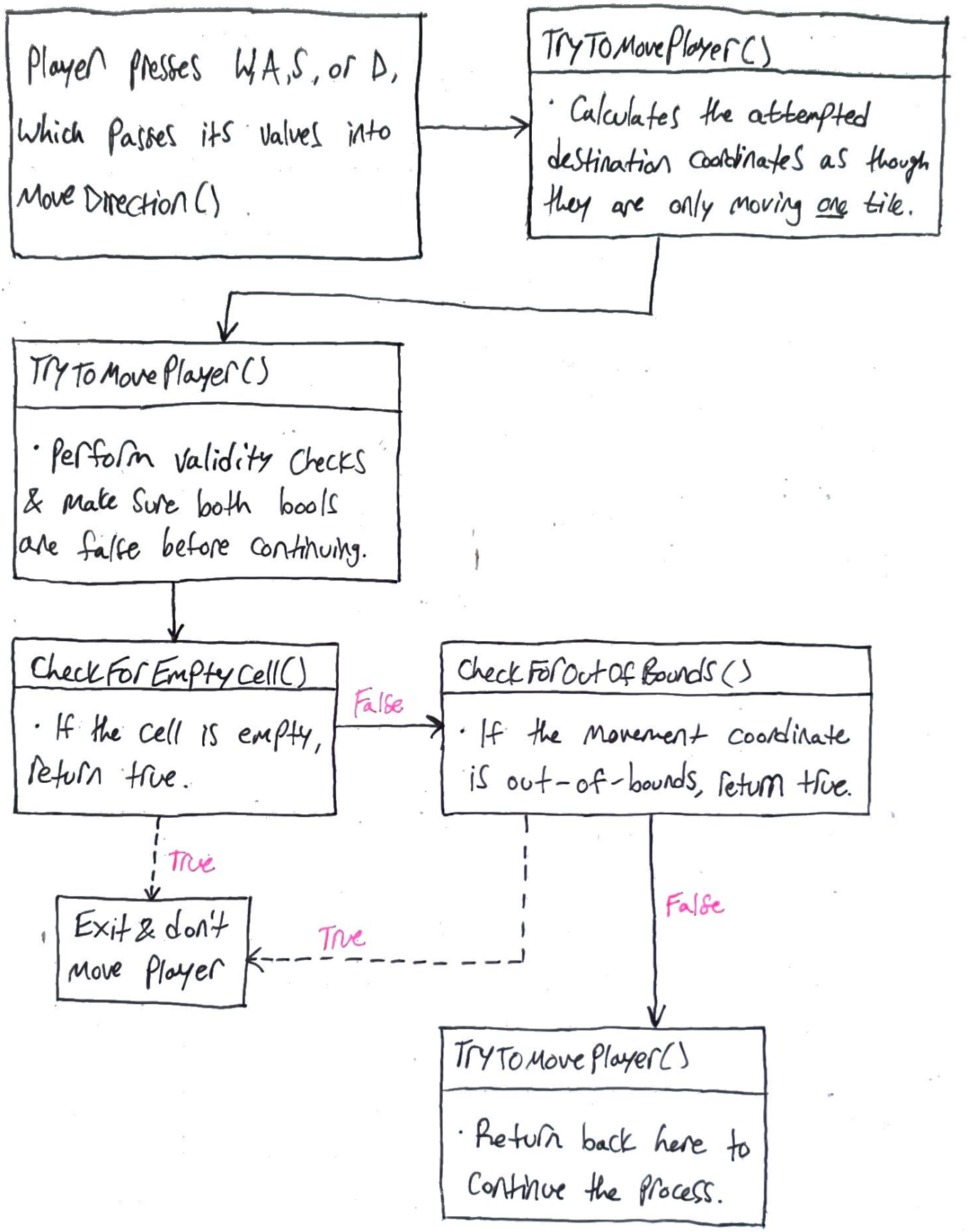
Understanding the flow of PlayerFixedMovement.cs 3/26/26

- I'm going to start with breaking down TryToMovePlayer().
- TryToMovePlayer() takes in two ints: xDir and zDir.
  - These directions are not coordinates, it's just identifying up, down, left, and right movement
- deltaX and deltaZ get the values of xDir and zDir.
- It appears we then add onto destinationX and destinationZ by the deltaX and deltaZ values
- I added some debug statements before & after this process of adding together values, and I realized it's actually being called twice.
  - Truthfully, it's probably being called for infinity, but it stops due to the out-of-bounds error on the second iteration.
- Because of this, I went ahead & created a separate method named MovePlayerToStartTile(), which is called at the end of UpdatePuzzleInformation().
  - this method skips all the delta & destination logic & just jumps straight to the SnapPlayerToTile() functionality.



(cont. on next page)

TryToMovePlayer()

· Now that we've validated that move, we're going to call CheckAttemptedTileType() into a switch-case statement.

```
Case TileType.Normal:
    NormalTile();
    break;

Case TileType.Ice:
    IceTile();
    break;
```

NormalTile()

· With a normal tile, we're going to want to move just <u>one</u> cell.

IceTile()

· With an ice tile, we want to recursively scan the next tile in the direction that we're moving in. We can do this by calling TryToMovePlayer() again, but we must ensure we are passing in the correct values for the directional parameters.

TryToMovePlayer(xDir, zDir)

These values don't define destination, they simply define the up, down, left, or right movement options for what directions are possible. The destination & validity checks occur within this method using these parameters. In the case of ice tiles, they're very much necessary for recursive checks in the <u>same direction</u>.